Miss Maya - Honest comparison

Replacing 12 regex output guards with one Qwen LLM-as-judge

Generated 2026-05-02 15:33 IST

Bottom line up front

Across 50 synthetic Maya sessions per arm (~435 Maya turns each), the LLM judge replaced all 12 deterministic regex guards. Conceptual rules - greetings, self-intros, fabricated biography, canned persona phrases - are caught BETTER by the judge: greeting-on-turn-2 regressions dropped from 32 to 15 sessions, mid-session self-intros went from 18 sessions to ZERO, and bio/soft-bio fabrications fell from 5 turns to ZERO. Mechanical rules - specifically the em-dash ban - regressed badly: the regex caught 100% of em-dashes (0 leaks), the judge let 363 turns through across all 50 sessions (82.9%). The two-question oscillation also regressed (3.2% -> 9.1%). Net: judge wins on intent, regex wins on punctuation. A 2-line deterministic post-strip would close the gap without re-introducing the regex layer.

What changed in code

* Removed: pg_apply_output_guard + 8 _PG_GUARD_* pattern lists (360 lines deleted from app.py)
* Added: judge_guard.py (~190 lines: prompt + adapter + JSON parser)
* Wired judge_review() into 5 call sites: chat(), pg_chat(), eval_50.call_maya()
* Pre-flight gate: 30 violations + 10 benign cases. Required >=90% recall, <=10% FP
* Pre-flight result: 100% recall (30/30), 10% FP (1/10 - and that 'FP' was a real soft-bio violation)

Per-category violation rates

Each row counts the number of Maya turns containing the violation, the number of distinct sessions affected, and the share of total Maya turns. Counts are run in the same scoring harness on both arms.

Category	Description	Before (regex)	After (judge)	
greeting_t2	Turn 2+ starts with 'Hi/Hello' + nam	32 t / 32 s (7.4%)	15 t / 15 s (3.4%)	v
self_intro_t2	'I'm Miss Maya' on returning user	18 t / 18 s (4.1%)	0 t / 0 s (0.0%)	v
multi_q	More than one '?' in a single reply	14 t / 8 s (3.2%)	40 t / 21 s (9.1%)	x
bio_hard	'I watched/listened/cooked X' fabric	1 t / 1 s (0.2%)	0 t / 0 s (0.0%)	v
soft_bio	'I follow cricket', 'I'm a tea perso	4 t / 3 s (0.9%)	0 t / 0 s (0.0%)	v
location_claim	'here in Bengaluru', 'I grew up in P	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
love_how_you	Surveillant openers	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
echo_praise	'You said X - perfect!'	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
canned_persona	'cardamom chai', 'old Hindi songs'	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
fake_quote	Quoting words user did not say	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
emoji	Any emoji codepoint	0 t / 0 s (0.0%)	0 t / 0 s (0.0%)	=
dashes	em/en-dash or hyphen between words	0 t / 0 s (0.0%)	363 t / 50 s (82.9%)	x

Total Maya turns scored: baseline 434, judge-only 438. 50 sessions per arm. Same scoring regex applied to both runs (consistent measurement).

Wins - where the judge beats the regex

These are categories where understanding intent matters more than pattern-matching. Regex catches what it was written for; the judge catches the family of variations the writer didn't anticipate.

self_intro_t2 **From 18 sessions to ZERO**

The regex hit 'I'm Miss Maya' literally. The judge understands the rule ('don't reintroduce yourself to a returning user') and catches paraphrases like 'I am your English chat partner' the regex missed.

greeting_t2 **32 sessions -> 15 sessions (~53% drop)**

The judge understands 'don't greet on turn 2+'. The regex was strict about 'Hi/Hello' but missed greeting-shaped openers ('Good to hear from you again, Aarti!'). Judge cleans both.

bio_hard **1 session -> 0 sessions**

Judge eliminated all 'I watched/listened/cooked X' fabrications. Marginal delta because baseline regex was already strong here, but verifies no regression.

soft_bio **3 sessions -> 0 sessions**

The 'I follow cricket / I'm a morning person' family of soft fabrications is gone. This was the category most likely to leak through the regex - the judge handles it cleanly after one prompt iteration.

Regressions - where the judge under-performs

Mechanical / surface-level rules don't need understanding. A regex catches them 100% of the time and zero false positives. The judge has to weigh them against every other rule in the prompt and tends to prioritize 'the reply still reads naturally' over 'punctuation is exactly right'.

dashes (em/en-dash) — 0 turns -> 363 turns (50/50 sessions, 82.9% of all Maya turns)

This is the headline regression. The regex stripped every em-dash by definition (a one-liner). The judge enumerates rule_10 in its violation list occasionally but doesn't actually rewrite the dashes out - the cost-benefit of rewriting a whole reply just to swap a dash for a comma is unfavorable to it. Fix: a single non-LLM substitution at the end (cleaned.replace(chr(8212), ',')) would close this gap entirely without resurrecting the regex layer.

01_A01_same_day T1: Hi Priyansh, I'm Miss Maya. It's a lovely evening to chat - how was your day?
01_A01_same_day T2: You worked on a project and watched a Bollywood movie - that sounds like a perfect balance. What movie did you watch?

multi_q (>1 question mark) — 14 turns / 8 sessions -> 40 turns / 21 sessions (3.2% -> 9.1%)

The regex used to strip extra '?' characters mechanically. The judge sees two questions and decides the reply still reads coherently, so it leaves them. This is the rule the judge weighs least aggressively because the prose flow argument competes with the rule. Same fix as dashes: a tiny mechanical post-step (if reply.count('?') > 1: keep only the last) would restore parity.

06_A06_same_day T3: The underdog spirit is always magnetic - it gives you that extra push, doesn't it? What are you rooting for next?
07_A07_same_day T2: "Iski baat kyun itni mushkil hai?" - it's a common challenge when mixing different worlds. The startup and Bollywood blend is a fresh idea. What part of the script keeps needing changes?

Architectural finding - librarian routing

The librarian is on Anthropic Haiku via the `claude` CLI - not Qwen.

call_llm_oneshot() in app.py prefers the Anthropic API client (Sonnet 4.6) if AnthropicAPIKey is set, otherwise falls back to: claude -p --model haiku. This is the function the librarian uses to merge each session into long-term memory.

Why this matters

Your stated preference was: keep everything on Qwen, including the librarian. The current code does NOT match that - the librarian falls through to the Haiku CLI on every session-end. During this 50-session run, the run log shows 10 '[pg-memory] LLM call FAILED' entries (the Haiku CLI failing transiently). The judge-only chat path works fine; the librarian background task is a separate concern that pre-dates today's changes.

Fix

Replace call_llm_oneshot() body with a Qwen-via-Bedrock call (same shape as the judge - one prompt in, one full string out). ~25 lines. Removes the Haiku dependency entirely and consolidates the model split to: Qwen for everything.

Recommendations

1. Close the dash + multi-q gap with 2 lines of post-processing

Not a backstop layer - just two surgical normalizations applied AFTER the judge: replace em-dash with comma, and if reply.count('?') > 1, keep only the last '?'. This restores parity on the two regressions without bringing back the 12-guard layer.

2. Move the librarian to Qwen

Rewrite call_llm_oneshot() to use stream_via_bedrock_qwen instead of the Haiku CLI. Removes the only remaining Anthropic dependency. Matches your stated 'use Qwen everywhere' direction.

3. Track judge confidence in production

judge_guard.py already returns confidence in its JSON. Log the confidence per Maya reply so we can spot drift if Qwen's behavior changes after a model update. If avg confidence drops below 0.85 over a rolling window, page someone.

4. Iterate the judge prompt only when categorical regressions appear

The biggest cost saving from this swap is that new violation modes are now prompt-only fixes - no code edits, no deploys. When a future eval surfaces a new failure mode, add a rule to JUDGE_PROMPT_TEMPLATE and re-run pre-flight.

Methodology

Two arms, both running the same QWEN-tuned prompt set. Same 50-session schedule:

Tier A	10 same-day sessions on one user (Priyansh)
Tier B	15 consecutive-day sessions on one user (Aarti)
Tier C	10 sporadic-gap sessions on one user (Rohan)
Tier D	10 cold-start sessions on 10 distinct fresh users
Tier E	5 deep-run weekly sessions on one user (Neha)

User responses are simulated by a separate model (Sonnet) playing each profile. Maya replies via Bedrock-Qwen. Both arms wrote into isolated memory directories so neither arm contaminated the other.

What we are not measuring

These rate-based numbers say nothing about whether Maya's replies feel WARM. The judge can be 100% correct on every rule and still produce stiff, transactional prose. Tone fidelity is a human-eval problem and would need a separate read-through. We also did not measure latency impact - the judge adds one extra Qwen round-trip (~0.7s observed in pre-flight) per Maya reply.